Structured and Object Oriented Programming

By

Dr. G.N.Balaji

Associate Professor, SCOPE

Introduction to C

- C is a high-level structured oriented programming language used in general purpose programming, developed by Dennis Ritchie at AT&T Bell labs, USA between 1969 and 1973.
- **Some Facts about C Programming Language**
- In 1988, the American National Standards Institute (ANSI) has formalized the C language.
- C was invented to write UNIX operating system.
- C is a successor of 'Basic Combined Programming Language' (BCPL) called B language.
- Linux OS, PHP and MySQL is written in C.
- C has been written in assembly language.

Uses of C Programming Language

- In the beginning C was used for developing system applications e.g. : Database Systems, Language Interpreters, Compilers and Assemblers, Operating Systems, Network Drivers, Word Processors

C has Become Very Popular for Various Reasons

- One of the early programming languages.
- Still the best programming language to learn quickly.
- C language is reliable, simple and easy to use.
- C language is a structured language.
- Modern programming concepts are based on C.
- It can be compiled on a variety of computer platforms.
- Universities preferred to add C programming in their courseware.

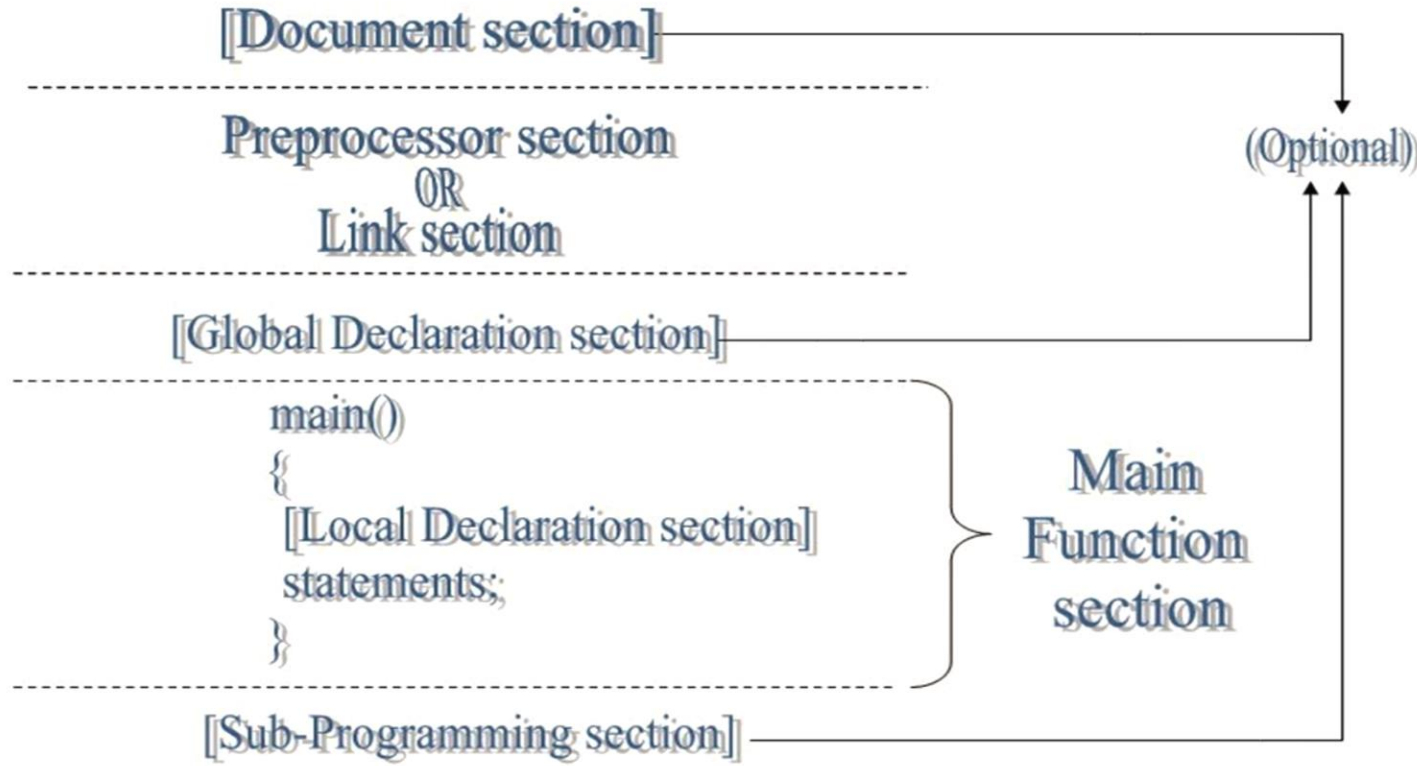
Features of C Programming Language

- C is a robust language with rich set of built-in functions and operators.
- Programs written in C are efficient and fast.
- C is highly portable, programs once written in C can be run on another machines with minor or no modification.
- C is basically a collection of C library functions; we can also create our own function and add it to the C library.
- C is easily extensible.

Structure of C Program

- Program comments- (optional)
- Preprocessor directives -(required)
- Global data type declarations- (optional)
- Named constants- (optional)
- Function declarations (prototypes) - (optional)
- main() -(required)
- Function definitions - (optional)

Basic structure of 'C' programme



1. Program Comments (Optional)

- /* and */ - Multi-line comments
- // - single line comment

2. Preprocessor Directives (Required)

- Lines that begin with a pound/hash sign #
- This symbol is used to add the content of a header file to a source program file. For example, #include <stdio.h>
- Programmer -defined header file surrounded by double quotation marks. #include “d:header1.h”
- Ex #define pi 3.17

3. Global Data Types Declarations (Optional)

Ex. int rollno;

double salary;

4. Named Constants (Optional)

Ex. const double salary = 10000.50;

is an identifier whose value is fixed and does not change during the execution of a program in which it appears.

5. Function declarations (Optional)

returntype functionname(list of parameters);

Ex. float add (float a, float b);

This function performs addition of two numbers and returns float result. It may be defined after the main function.

6. main () – (Required)

Each C program must have one main() function.

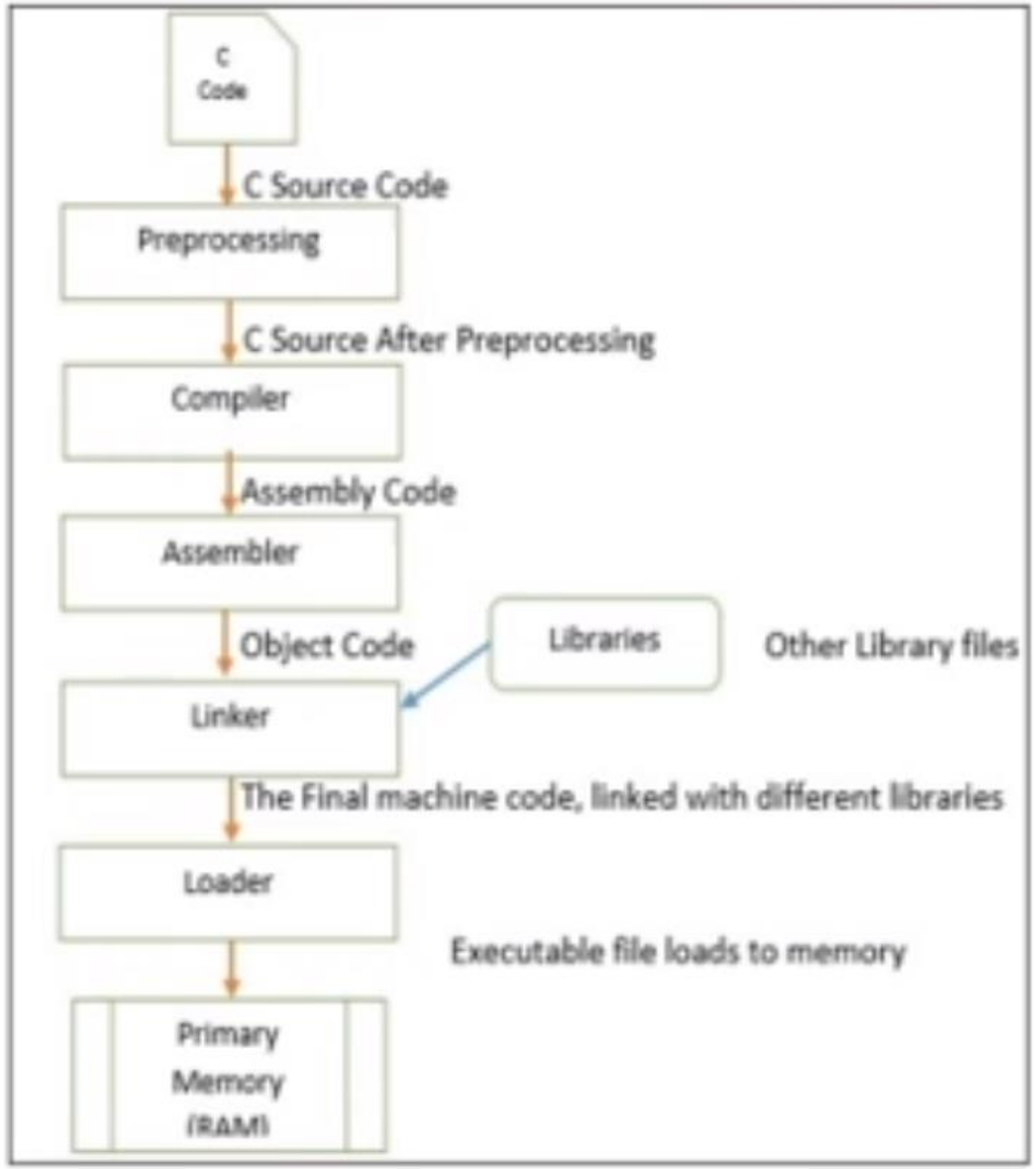
```
int main()
 {
    Statement;
    Statement;
    function call;
    return 0;
 }
```

7. Function definitions (Optional)

Block of code that performs a specific task

- A statement is a specification of an action to be taken by the computer.
- Compound Statements is a list of statements enclosed in braces, { }.
- For successful execution of the program, it return constant value 0.
- The datatype specifier for other functions can be int, double, char, void (no return value), and so on, depending on the type of data that it returns.

Execution of a Program



C Code – source code with extension .c This code is sent to the Preprocessors section.

Preprocessing – In this section the preprocessor files such as include and define are attached with our code.

Compiler – After generating the preprocessed source code, it moves to the compiler and the compiler generates the assembly level code.

Assembler – This part takes the assembly level language from compiler and generates the Object code, this code is quite similar to the machine code (set of binary digits).

Linker – It takes the object code and link it with other library files, these library files are not the part of our code, but it helps to execute the total program. After linking the Linker generates the final Machine code which is ready to execute.

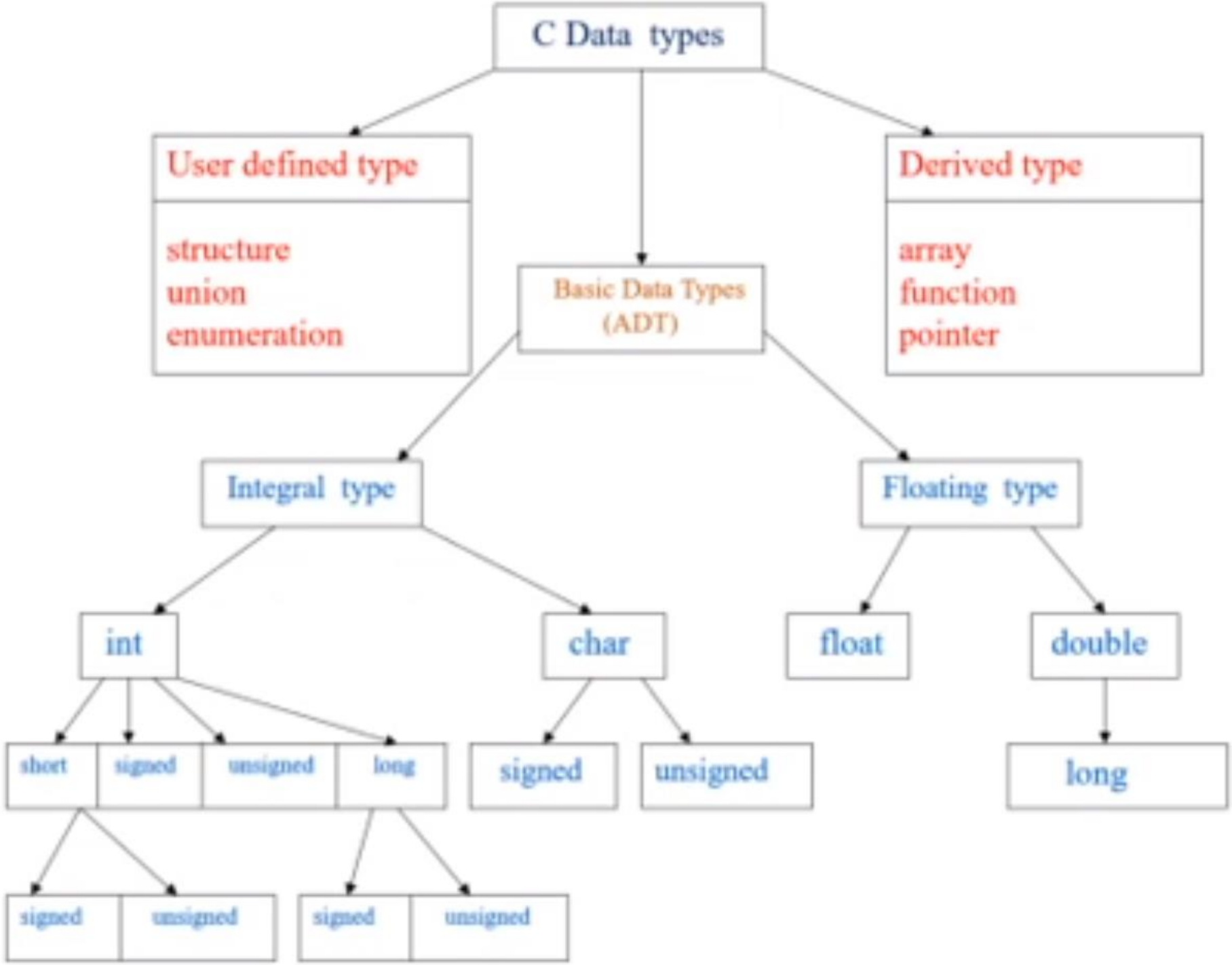
Loader – A program, will not be executed until it is not loaded in to Primary memory. Loader helps to load the machine code to RAM and helps to execute it. While executing the program is named as Process.

Data Type

- The data-type in a programming language is the collection of data with values having fixed meaning as well as characteristics. Some of them are an integer, floating point, character, etc. Usually, programming languages specify the range values for given data-type.
- A data-type in C programming is a set of values and is determined to act on those values. C provides various types of data-types which allow the programmer to select the appropriate type for the variable to set its value.

C Data Types are used to:

- Identify the type of a variable when it declared.
- Identify the type of the return value of a function.
- Identify the type of a parameter expected by a function.


C Data types
User defined type
structure
union
enumeration
Basic Data Types
(ADT)
Derived type
array
function
pointer
Integral type
Floating type
int
char
float
double
short
signed
unsigned
long
signed
unsigned
long
signed
unsigned
signed
unsigned

Memory

- Bit - 0 or 1
- Byte – Sequence of 8 bits- 2^8- 256 - Range(0 to 255)
  - Each character represented by Byte (ASCII)
  - Ex. A – ASCII value is 65D- 0100 0001B – 41H
- 1KB - Sequence of 10 bits- 2^{10}- 1024 - Range(0 to 1023)
- 1MB – 1024KB - 2^{20}
- 1GB – 1024MB - 2^{30}
- 1TB – 1024GB - 2^{40}

Signed and Unsigned Number

Signed	Unsigned
Positive and Negative	Positive only
1 bit reserved for sign and remaining bits for magnitude	No sign bit and all bits for magnitude
Ex. 8 bit number – 1 bit reserved for sign – Most Significant Bit (MSB) • MSB – 0 for +ve, 1 for –ve numbers • 7 bits for magnitude • 2^7 - 128 • Range is -2^7 to $2^7 - 1$ => -128 to 127 0000 0000 -> 0 1000 0001 -> -1 0000 0001 -> 1 1000 0010 -> -2 : : 0111 1111 -> 127 1111 1111 -> -127	Ex. 8 bit number – 2^8 – 256 – (0-255) 0000 0000 - 0 0000 0001 - 1 : 1111 1111 - 255

Limits of Data Types

Type	Bytes	Range
signed short int / short int	2	-32768 to 32767
unsigned short int	2	0 to 65535
signed int / int	2	-32768 to 32767
unsigned int	2	0 to 65535
long int / signed long int	4	-2^{31} to $2^{31}-1$
unsigned long int	4	0 to $2^{32}-1$
char / signed char	1	-128 to 127
unsigned char	1	0 to 255
float	4	$1.17\ e^{-38}$ to $3.4\ e^{38}$
double	8	$2.22\ e^{-308}$ to $1.79\ e^{308}$
long double	10	$1.1e^{-4932}$ to $1.18\ e^{4932}$

Advantages of C

- C is the building block for many other programming languages.
- Programs written in C are highly portable.
- Several standard functions are there (like in-built) that can be used to develop programs.
- C programs are basically collections of C library functions, and it's also easy to add own functions in to the C library.
- The modular structure makes code debugging, maintenance and testing easier.

Disadvantages of C

- C does not provide Object Oriented Programming (OOP) concepts.
- There is no concept of Namespace in C.
- C does not provide binding or wrapping up of data in a single unit.
- C does not provide Constructor and Destructor.

printf() and scanf() in C

- The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).
- printf() function
- The **printf() function** is used for output. It prints the given statement to the console.

Syntax:

```
printf("format string",argument_list);
```

- The printf() is a library function to send formatted output to the screen. The function prints the string inside quotations.
- To use printf() in our program, we need to include stdio.h header file using the #include <stdio.h> statement.

Sample Program

```
#include <stdio.h>
int main()
{
    printf("Hello World");
   return 0;
}
```

Format Specifiers

- The format specifiers are used in C for input and output purposes. Using this concept, the compiler can understand that what type of data is in a variable during taking input using the scanf() function and printing using printf() function.

Format Specifiers

Data Type		Format
Integer	Integer	%d
	Short	%d
	Short unsigned	%u
	Long	%ld
	Long assigned	%lu
	Hexadecimal	%x
	Long hexadecimal	%lx
	Octal	%O (letter 0)
	long octal	%lo
Real	float,double	%f, %lf, %g
Character		%c
String		%s

scanf()

- In C programming, scanf() is one of the commonly used function to take input from the user. The scanf() function reads formatted input from the standard input such as keyboards.

```
#include <stdio.h>
int main()
{
   int testInteger;
   printf("Enter an integer: ");
   scanf("%d", &testInteger);
   printf("Number = %d",testInteger);
   return 0;
}
```

Constants

- A constant is an entity whose value can't be changed during the execution of a program.
- Constants are classified as:
    1. Literal constants
    2. Qualified constants
    3. Symbolic constants

Defining Constants

- There are two simple ways in C to define constants −
- Using **#define** preprocessor.
- Using **const** keyword.

Literal Constants

- C language supports the following five major types of Literal

1. Integer constant
2. Real constant
3. Character constant – represented in single quotes ‘x’, ‘1’, ‘-’
4. String constant - represented in double quotes “hai”, “123hello”
5. Backslash character constant – ‘\n’, ‘\t’, ‘\r’

Integer Constants

- Three types - decimal integer, octal integer and hexadecimal integer.
- Decimal integers consists of a set of digits from 0 to 9, preceded by an optional minus sign. Empty spaces, commas and non-digit characters are not permitted between digits.

Ex: 1010 -2020

```
#include<stdio.h>
void main()
{
int a=20;
printf("%d",a);
}
```

Output:

20

Integer Constants

- Three types - decimal integer, octal integer and hexadecimal integer.
- Decimal integers consists of a set of digits from 0 to 9, preceded by an optional minus sign. Empty spaces, commas and non-digit characters are not permitted between digits.

Ex: 1010 -2020

```
#include<stdio.h>
void main()
{
int a=20;
printf("%d",a);
}
```

Output:

20

- An octal integer combination of digits from the set 0 to 7, with a leading 0.
- Empty spaces, commas and non-digit characters are not permitted between digits.

Ex. 047 0 0665

```
#include<stdio.h>
void main()
{
int a=020;          // a is an octal integer
printf("%d",a);     // print its decimal value
}
```

Output:

16

5

Real Constants

- Real (floating point) constant may be represented by decimal notation.
- The number may not have digits before the decimal point & after the decimal point.

  Ex: 200. .99 -.67 17.67
- Real constants may also be expressed in exponential or scientific notation.
- For example, the value 345.67 may be written as 3.4567e2 in exponential notation. e2 means multiply by 10^2. The general form is

  mantissa e exponent
- The mantissa is either a real number expressed in decimal notation or an integer.
- The exponent should be an integer with an optional plus or minus sign.
- The letter e separating the mantissa and the exponent can be written in either lowercase or uppercase.

  Ex: 0.77e3 20e-1 245E+5

Integer Constants

- A sequence of digits preceded by 0x or 0X is considered as hexadecimal integer (hex integer). They may also include alphabets A to F or a to f. The letter A to F represents the numbers 10 to 15.
- Empty spaces, commas and non-digit characters are not permitted between digits.

  Ex. 0x7 0X567A 0xabc

```
#include<stdio.h>
void main()
{
int a=0x20;      // a is a hexadecimal integer
printf("%d",a); // print its decimal value
}
```

Output:

32

```
#include<stdio.h>
void main()
{
int a=0x1F;
printf("%d",a);
}
```

Output:

31

Real Constants

```
#include<stdio.h>
void main()
{
   float f=345.6789;
   printf("Floting format = %f\n",f);
   printf("Exponential format = %e\n",f);
   printf("Exponential Format = %E\n",f);
}
```

```
Floting format = 345.678894
Exponential format = 3.456789e+02
Exponential Format = 3.456789E+02
```

Qualified Constants

```
#include<stdio.h>
void main()
{
  const float f=345.6789;
  const int a =200;
  printf(“Integer Constant = %d\n”,a);
  printf("Floting format = %f\n",f);
  printf("Exponential format = %e\n",f);
  printf("Exponential Format = %E\n",f);
  a=a+10;
}
```

Error
cannot modify const object

Symbolic Constants

- Symbolic constants are created with the help of the define preprocessor directive.
- For ex #define PI= 3.14 defines PI as a symbolic constant with the value 3.14.
- Each symbolic constant is replaced by its actual value during the pre-processing stage.

```
#include<stdio.h>
#define pi 3.1417
void main()
{
  float a,r=10;
  a=pi*r*r;
  printf("Area of circle is =%f\n",a);
}
```

Output:
Area of circle is = 314.170013

Enumeration Constant

- It is a user-defined data type.
- It consists of integer constant values.

- Syntax 1 :

```
enum udfn {variable1, variable2, … variablen};
```

This is equivalent to

```
const variable1= 0;
const variable2= 1;
:
const variablen= n-1;
```

- Syntax 2 :

```
enum udfn {variable1=value1, variable2=value2, … variablen=valuen};
```

Assign values to variables explicitly

Ex. enum marks{m1=60, m2=75};

Program 1

```
#include<stdio.h>
void main()
{
   enum days{sun,mon,tue,wed,thu,fri,sat};
   printf("Sunday = %d\n",sun);
   printf("Monday = %d\n",mon);
   printf("Tuesday = %d\n",tue);
   printf("Wednesday = %d\n",wed);
   printf("Thursday = %d\n",thu);
   printf("Friday = %d\n",fri);
   printf("Saturday = %d\n",sat);
}
```

Output:

```
Sunday = 0
Monday = 1
Tuesday = 2
Wednesday = 3
Thursday = 4
Friday = 5
Saturday = 6
```

Program 3

```
#include<stdio.h>
enum days{sun,mon,tue=5,wed,thu,fri=20,sat};
void main()
{
  printf("Sunday = %d\n",sun);
  printf("Monday = %d\n",mon);
  printf("Tuesday = %d\n",tue);
  printf("Wednesday = %d\n",wed);
  printf("Thursday = %d\n",thu);
  printf("Friday = %d\n",fri);
  printf("Saturday = %d\n",sat);
}
```

Output:

Sunday = 0

Monday = 1

Tuesday = 5

Wednesday = 6

Thursday = 7

Friday = 20

Saturday = 21

Program 4 – Negative numbers are permitted

```
#include<stdio.h>
enum days{sun,mon,tue=-1,wed,thu,fri=20,sat};
void main()
{
   printf("Sunday = %d\n",sun);
   printf("Monday = %d\n",mon);
   printf("Tuesday = %d\n",tue);
   printf("Wednesday = %d\n",wed);
   printf("Thursday = %d\n",thu);
   printf("Friday = %d\n",fri);
   printf("Saturday = %d\n",sat);
}
```

Output:

```
Sunday = 0
Monday = 1
Tuesday = -1
Wednesday = 0
Thursday = 1
Friday = 20
Saturday = 21
```

Keywords in C

- Keywords are predefined, reserved words in C language and each of which is associated with specific features. These words help us to use the functionality of C language. They have special meaning to the compilers.

Keywords

Keywords in C Programming

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Rules for defining variables

- A **variable** is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.
- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid and invalid variables

1.int a;
2.int _ab;
3.int a30;

1.int 2;
2.int a b;
3.int long;

Operators in C

- An operator is a symbol used to do a mathematical or logical manipulations.

1. Arithmetic operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increments and Decrement Operators
6. Conditional Operators
7. Bitwise Operators
8. Special Operators

Arithmetic Operators

Operator	Meaning
+	Addition or Unary Plus
–	Subtraction or Unary Minus
*	Multiplication
/	Division
%	Modulus Operator

Arithmetic Operations

- **Integer Arithmetic**

Let x = 27 and y = 5

x + y = 32 x – y = 22 x * y = 115

x % y = 2 x / y = 5

- **Floating point Arithmetic**

Let x = 14.0 and y = 4.0 then

x + y = 18.0 x – y = 10.0

x * y = 56.0

- **Mixed mode Arithmetic**

15/10.0 = 1.5

Sample program

```
#include<stdio.h> // Header File
int b=10; //Global Declaration
void main ( ) /* main is the starting of every c program */
{
int a,c;  //Local Declaration
scanf(“%d”,&a);
printf(“ \n The sum of the two values:”);
c = a+b;
printf(“%d”,c);
}
```

Division operator on Different Data Type

Operation	Result	Example
int/int	int	5/2 = 2
int/real	real	5/2.0 = 2.5
real/int	real	5.0/2 = 2.5
real/real	real	5.0/2.0 = 2.5

Sample program

```
#include<stdio.h>
void main ( )
{
int a=10,b=4,c;
float d=3,e;
c = a/b;
printf(" \n value a/b is:%d",c);
e = a/d;
printf("\n value a/d is:%f",e);
}
```

Output

value a/b is:2

value a/d is:3.333333

Relational Operators

- It is used to **compare** two or more operands.
- 5 < 9 which will return 1

Operator	Meaning
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to

Logical Operators

- It is used to **combine** the result of two or more condition.
- Eg: (i>10)&&(j>5).
- (i>10)||(j>5) etc,.

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Sample program

```
#include<stdio.h>
#include <conio.h>
void main ( )
{
int a=10,b=3,c=5,e;
   if(a>b)        // relational operator
   {
            printf(" \n a is bigger than b");
   }
   if((a>b)&&(a>c))          //Logical operator
   {
            printf(" \n a is biggest");
   }
}
```

Output

a is bigger than b

a is biggest

Assignment Operators

It is used to **assign a value or expression** etc to a variable.

Compound operator

It is also used to assign a value to a variable.

Eg: x + = y means x = x + y

Nested operator

It is used for multiple assignment. Eg: i = j = k = 0;

Statement with simple assignment operator	**Statement with shorthand operator**
a = a + 1	a += 1
a = a – 1	a -= 1
a = a * (n+1)	a *= (n+1)
a = a / (n+1)	a /= (n+1)
a = a % b	a %= b

Increment and Decrement Operators

1. ++ variable name (Pre Increment)
2. variable name++ (Post Increment)
3. – –variable name (Pre Decrement)
4. variable name– – (Post Decrement)

Consider the following

```
m = 5;
y = ++m; (prefix)
```

value of y and m would be 6

```
m = 5;
y = m++; (post fix)
```

value of y will be 5 and that of m will be 6.

Sample Program

```
#include<stdio.h>
#include <conio.h>
void main ( )
{
int a=5;
clrscr( );
printf(" \n Post increment Value:%d",a++);
printf(" \n Pre increment Value:%d",++a);
printf(" \n Pre decrement Value:%d",--a);
printf(" \n Post decrement Value:%d",a--);
getch( );
}
```

Output

Post increment Value:5

Pre increment Value:7

Pre decrement Value:6

Post decrement Value:6

Conditional or Ternary Operator

It is used to checks the condition and execute the statement depending on the condition.

The conditional operator consists of 2 symbols the question mark (?) and the colon (:)

syntax:

exp1 ? exp2 : exp3

```
 a = 10;
b = 15;
x = (a > b) ? a : b
```

- For example

a=10 b=15

x=(a>b)?a:b

In this sample x will be assigned the value of b.

```
if(a>b)                 if(10>15)
   x=a;                    x=10;
else                    else
   x=b;                    x=15;
```

```c
#include <stdio.h>

int main() {

  // create variables
  char operator = '+';
  int num1 = 8;
  int num2 = 7;

  // using variables in ternary operator
  int result = (operator == '+') ? (num1 + num2) : (num1 - num2);
  printf("%d", result);

  return 0;
}
```

Sample Program

```
#include<stdio.h>
#include <conio.h>
void main ( )
{
int a=5,b=8,c;
clrscr( );
c = a>b?a:b;    //Conditional operator
printf(" \n The Larger Value is%d",c);
getch( );
}
```

Output: The Larger Value is 8

Bitwise Operators

- It is used to manipulate data at bit level.
- Eg: a=5 i.e 0000 0101

  b=4 i.e 0000 0100

  Then a & b = 0000 0100

  a | b = 0000 0101 etc,.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise Exclusive
<<	Shift left
>>	Shift right

Sample program

```
#include<stdio.h>
#include <conio.h>
void main ( )
{
int a=5,b=4,c;
//char a=5,b=4,c;
clrscr( );
c = a&b;
printf(" \n value a&b is:%d",c);
getch( );
}
```

Output: value a&b is:4

Special Operator

- comma operator (,)
- sizeof operator
- pointer operator (& and *)
- member selection operators (. and ->).

The Comma Operator -to link related expressions together. Eg: int x=5,y=10;

Contd…sizeof

```
#include<stdio.h>
#include <conio.h>
void main ( )
{
int c;
clrscr( );
printf(" \n size of int is:%d",sizeof(c));
getch( );
}
```

Output: size of int is: 2

Operator Precedence & Associativity

- The arithmetic expressions evaluation are carried out based on the precedence and associativity.
- The evaluation are carried in two phases.
  - **First Phase:** High Priority operators are evaluated.
  - **Second Phase:** Low Priority operators are evaluated.

Contd…

Precedence	Operator
High	* , / , %
Low	+ , -

Operator Precedence and Associativity

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Program

```
// Task is (a+b)*(c+d)
#include<stdio.h>
void main()
{
   int a=10,b=20, c=30,d=5,r;
   r=a+b*c+d;
   printf("Incorrect Result is = %d\n",r);
   r=(a+b)*(c+d);
   printf("Correct Result is = %d\n",r);
}
```

Output:

```
Incorrect Result is = 615
Correct Result is = 1050
```

Expressions

- An expression is a sequence of operators and operands that specifies computation of a value.
  - Ex. a=152*10 is an expression with three operands a,152,10 and 2 operators + and =
- An expression that has only one operator is known as a simple expression.
  - Ex: a++
- An expression that involves more than one operator is called a compound expression.
  - Ex: b=10+(3*5)

Branching /Conditional/Selection statements

- It can be classified into two major types.
  1. if-else statements 2. switch-case statements

if-else statements:

- if is a powerful decision making statement which is used to control the sequence of the execution of statements.
- Syntax:

```
if(expression or condition) //true then compiler executes this block
{
      Statement-1;
      Statement -n;
}
else   //false then compiler executes this block
 {
      Statement-1;
      Statement -n;
 }
```

Program – if else

```
#include<stdio.h>
void  main()
{
      float salary, bonus;
      printf("Enter  your  Salary: ");
      scanf(“%f”, &salary);
      if(salary>=15000)
            bonus=salary*0.2;
      else
            bonus=salary*0.3;
      printf("Bonus: %f”, bonus);
}
```

Run1:
Enter your Salary: 20000
Bonus: 4000.0
Run2:
Enter your Salary: 5000
Bonus: 1500.0

Nested If

```
if(test_expression one)
{
   if(test_expression two) {
      //Statement block Executes when the boolean test expression two is true.
   }
}
else
{
    //else statement block
}
```

else if

```
if(test_expression)
{
  //execute your code
}
else if(test_expression n)
{
  //execute your code
}
else
{
  //execute your code
}
```

switch case

- It is a multiple branching statement.
- Based on a condition, the control is transferred to one of the many possible branches.
- Syntax:

```
switch(label)
{
case label:
        statement ;
        break;
case label:
        statement ;
        break;
default:
        statement ;
}
```

- Label should be an integer or character constants(Entered in single quotes- 'a').
- Case label must not be floating point numbers (10.5).
- Case Label must not be a string constants("Monday").
- Case label cannot be an expression('a+b').

- The switch label may be any valid expression including the value of the variable / an arithmetic expression / logical comparison / bitwise expression / the return value from a function call.
- The switch label value is checked against each of the specified case labels.
- If it matches, the statement following that case label will be executed. If no match found, the statement following the default case is executed.

```
#include<stdio.h>
void  main()
{
int a, b, ch;
printf("Enter two integer inputs");
scanf(“%d%d”,&a,&b);
printf("1.Addition\n2.Subtraction");
printf("3.Multiplication\n4.Division\n");
printf("Enter your option: ");
scanf(“%d”,&ch);
switch(ch)
      {
      case 1:   printf("Addition %d”,(a+b));  break;
      case 2:   printf("Subtraction %d“,(a-b));  break;
      case 3:   printf("Multiplication %d“,(a*b)); break;
      case 4:   printf("Division %d“, (a/b)); break;
      default: printf(“ Invalid choice”);
      }
}
```

```
Run1:
Enter  two integer inputs
10
5
1.Addition
2.Subtraction
3.Multiplication
4.Division
Enter your option: 1
Addition15
Run2:
Enter  two integer inputs
30
6
1.Addition
2.Subtraction
3.Multiplication
4.Division
Enter your option: 4
Division 5
```

Unconditional Branching Statements

- goto
- Syntax:

```
goto label;
// label is the position where the control
is to be transferred
```

```
// Program to calculate the sum and average of positive numbers
// If the user enters a negative number, the sum and average are displayed.

#include <stdio.h>

int main() {

  const int maxInput = 100;
  int i;
  double number, average, sum = 0.0;

  for (i = 1; i <= maxInput; ++i) {
    printf("%d. Enter a number: ", i);
    scanf("%lf", &number);
// go to jump if the user enters a negative number
    if (number < 0.0) {
      goto jump;
    }
    sum += number;
  }

jump:
  average = sum / (i - 1);
  printf("Sum = %.2f\n", sum);
  printf("Average = %.2f", average);

  return 0;
}
```

Control Statements

- Control statements can be classified into two types.
  1. Branching/ Decision making statements
  2. Loop statements
- Branching statements are used to transfer program control from one point to another.
- a) Conditional Branching:- Program control is transferred from one point to another based on the result of some condition
  - if, if-else, switch
- b) Unconditional Branching:- Program control is transferred from one point to another without checking any condition
  - goto, break, continue, return